

pm_lab3
1.0

Создано системой Doxygen 1.16.1

1 Сравнение модифицированных генераторов случайных чисел	1
2 Дерево директорий	3
2.1 Алфавитный указатель директорий	3
3 Алфавитный указатель пространств имен	5
3.1 Пространства имен	5
4 Алфавитный указатель классов	7
4.1 Классы	7
5 Список файлов	9
5.1 Файлы	9
6 Директории	11
6.1 Содержание директории build/CMakeFiles/4.3.1	11
6.2 Содержание директории build	11
6.3 Содержание директории build/CMakeFiles	11
6.4 Содержание директории build/CMakeFiles/4.3.1/CompilerIdC	11
6.5 Содержание директории build/CMakeFiles/4.3.1/CompilerIdCXX	11
6.6 Содержание директории scripts	12
6.7 Содержание директории src	12
7 Пространства имен	13
7.1 Пространство имен plot_distribution	13
7.1.1 Переменные	13
7.1.1.1 bins	13
7.1.1.2 data	13
7.1.1.3 dpi	13
7.1.1.4 figsize	13
7.1.1.5 n	14
7.2 Пространство имен plot_timing	14
7.2.1 Переменные	14
7.2.1.1 df	14
7.2.1.2 dpi	14
7.2.1.3 figsize	14
7.2.1.4 label	14
7.2.1.5 marker	14
7.2.1.6 subset	14
8 Классы	15
8.1 Класс ModifiedLCG	15
8.1.1 Подробное описание	15
8.1.2 Конструктор(ы)	15
8.1.2.1 ModifiedLCG()	15
8.1.3 Методы	16

8.1.3.1 next()	16
8.1.3.2 nextDouble()	16
8.2 Класс ModifiedMiddleSquare	16
8.2.1 Подробное описание	16
8.2.2 Конструктор(ы)	17
8.2.2.1 ModifiedMiddleSquare()	17
8.2.3 Методы	17
8.2.3.1 next()	17
8.2.3.2 nextDouble()	17
8.3 Класс ModifiedXorShift	17
8.3.1 Подробное описание	18
8.3.2 Конструктор(ы)	18
8.3.2.1 ModifiedXorShift()	18
8.3.3 Методы	18
8.3.3.1 next()	18
8.3.3.2 nextDouble()	18
8.4 Структура SampleStats	18
8.4.1 Подробное описание	19
8.4.2 Данные класса	19
8.4.2.1 cv	19
8.4.2.2 mean	19
8.4.2.3 stddev	19
8.4.2.4 variance	19
8.5 Структура TestResult	20
8.5.1 Подробное описание	20
8.5.2 Данные класса	20
8.5.2.1 passed	20
8.5.2.2 pvalue	20
8.5.2.3 statistic	20
8.6 Класс Timer	20
8.6.1 Подробное описание	21
8.6.2 Конструктор(ы)	21
8.6.2.1 Timer()	21
8.6.2.2 ~Timer()	21
8.6.3 Методы	21
8.6.3.1 elapsed()	21
9 Файлы	23
9.1 Файл build/CMakeFiles/4.3.1/CompilerIdC/apple-sdk.c	23
9.2 Файл build/CMakeFiles/4.3.1/CompilerIdC/CMakeCCompilerId.c	23
9.2.1 Макросы	24
9.2.1.1 __has_include	24
9.2.1.2 ARCHITECTURE_ID	24

9.2.1.3 C_STD_11	24
9.2.1.4 C_STD_17	24
9.2.1.5 C_STD_23	24
9.2.1.6 C_STD_99	24
9.2.1.7 C_VERSION	24
9.2.1.8 COMPILER_ID	24
9.2.1.9 DEC	24
9.2.1.10 HEX	25
9.2.1.11 PLATFORM_ID	25
9.2.1.12 STRINGIFY	25
9.2.1.13 STRINGIFY_HELPER	25
9.2.2 Функции	25
9.2.2.1 main()	25
9.2.3 Переменные	25
9.2.3.1 info_arch	25
9.2.3.2 info_compiler	26
9.2.3.3 info_language_extensions_default	26
9.2.3.4 info_language_standard_default	26
9.2.3.5 info_platform	26
9.3 Файл build/CMakeFiles/4.3.1/CompilerIdCXX/apple-sdk.cpp	26
9.4 Файл build/CMakeFiles/4.3.1/CompilerIdCXX/CMakeCXXCompilerId.cpp	26
9.4.1 Макросы	27
9.4.1.1 __has_include	27
9.4.1.2 ARCHITECTURE_ID	27
9.4.1.3 COMPILER_ID	27
9.4.1.4 CXX_STD	27
9.4.1.5 CXX_STD_11	27
9.4.1.6 CXX_STD_14	27
9.4.1.7 CXX_STD_17	27
9.4.1.8 CXX_STD_20	28
9.4.1.9 CXX_STD_23	28
9.4.1.10 CXX_STD_98	28
9.4.1.11 DEC	28
9.4.1.12 HEX	28
9.4.1.13 PLATFORM_ID	28
9.4.1.14 STRINGIFY	28
9.4.1.15 STRINGIFY_HELPER	29
9.4.2 Функции	29
9.4.2.1 main()	29
9.4.3 Переменные	29
9.4.3.1 info_arch	29
9.4.3.2 info_compiler	29
9.4.3.3 info_language_extensions_default	29

9.4.3.4 info_language_standard_default	29
9.4.3.5 info_platform	30
9.5 Файл scripts/plot_distribution.py	30
9.6 Файл scripts/plot_timing.py	30
9.7 Файл src/Benchmark.h	30
9.7.1 Подробное описание	31
9.7.2 Функции	31
9.7.2.1 benchmark()	31
9.8 Benchmark.h	31
9.9 Файл src/main.cpp	32
9.9.1 Макросы	32
9.9.1.1 _USE_MATH_DEFINES	32
9.9.2 Функции	32
9.9.2.1 benchmarkMt19937()	32
9.9.2.2 generateSample()	32
9.9.2.3 main()	33
9.9.2.4 printHeader()	33
9.9.2.5 testGenerator()	33
9.10 Файл src/ModifiedLCG.cpp	33
9.11 Файл src/ModifiedLCG.h	33
9.11.1 Подробное описание	33
9.12 ModifiedLCG.h	34
9.13 Файл src/ModifiedMiddleSquare.cpp	34
9.14 Файл src/ModifiedMiddleSquare.h	34
9.14.1 Подробное описание	34
9.15 ModifiedMiddleSquare.h	35
9.16 Файл src/ModifiedXorShift.cpp	35
9.17 Файл src/ModifiedXorShift.h	35
9.17.1 Подробное описание	35
9.18 ModifiedXorShift.h	35
9.19 Файл src/RandomTests.cpp	36
9.19.1 Функции	36
9.19.1.1 chiSquareTest()	36
9.19.1.2 minimumDistanceTest()	36
9.19.1.3 monkeyTest()	36
9.19.1.4 overlappingSumsTest()	37
9.19.1.5 runsTest()	37
9.20 Файл src/RandomTests.h	37
9.20.1 Подробное описание	37
9.20.2 Функции	38
9.20.2.1 chiSquareTest()	38
9.20.2.2 minimumDistanceTest()	38
9.20.2.3 monkeyTest()	38

9.20.2.4 overlappingSumsTest()	38
9.20.2.5 runsTest()	38
9.20.2.6 squeezeTest()	39
9.21 RandomTests.h	40
9.22 Файл src/Statistics.cpp	40
9.22.1 Функции	41
9.22.1.1 computeStats()	41
9.23 Файл src/Statistics.h	41
9.23.1 Подробное описание	41
9.23.2 Функции	41
9.23.2.1 computeStats()	41
9.24 Statistics.h	42
9.25 Файл src/Timer.cpp	42
9.25.1 Подробное описание	42
9.26 Файл src/Timer.h	42
9.27 Timer.h	42
Предметный указатель	43

Глава 1

Сравнение модифицированных генераторов случайных чисел

Проект реализует три генератора:

- [ModifiedLCG](#) – линейный конгруэнтный с 64-битным состоянием
- [ModifiedXorShift](#) – XorShift128 с добавлением нелинейности
- [ModifiedMiddleSquare](#) – модифицированный метод середины квадрата

Для каждого генератора:

- Генерируются выборки разного объёма (от 1000 до 1000000)
- Выполняется семь статистических тестов (хи-квадрат, серий, перекрывающихся сумм, минимального расстояния, "обезьяны", "сжатия", а также замеряется производительность).
- Результаты сохраняются в CSV-файл и выводятся в консоль.

Автор

Ваше Имя

Дата

Год

Глава 2

Дерево директорий

2.1 Алфавитный указатель директорий

4.3.1	11
CompilerIdC	11
apple-sdk.c	23
CMakeCCompilerId.c	23
CompilerIdCXX	11
apple-sdk.cpp	26
CMakeCXXCompilerId.cpp	26
build	11
CMakeFiles	11
4.3.1	11
CompilerIdC	11
apple-sdk.c	23
CMakeCCompilerId.c	23
CompilerIdCXX	11
apple-sdk.cpp	26
CMakeCXXCompilerId.cpp	26
CMakeFiles	11
4.3.1	11
CompilerIdC	11
apple-sdk.c	23
CMakeCCompilerId.c	23
CompilerIdCXX	11
apple-sdk.cpp	26
CMakeCXXCompilerId.cpp	26
CompilerIdC	11
apple-sdk.c	23
CMakeCCompilerId.c	23
CompilerIdCXX	11
apple-sdk.cpp	26
CMakeCXXCompilerId.cpp	26
scripts	12
plot_distribution.py	30
plot_timing.py	30
src	12
Benchmark.h	30

main.cpp	32
ModifiedLCG.cpp	33
ModifiedLCG.h	33
ModifiedMiddleSquare.cpp	34
ModifiedMiddleSquare.h	34
ModifiedXorShift.cpp	35
ModifiedXorShift.h	35
RandomTests.cpp	36
RandomTests.h	37
Statistics.cpp	40
Statistics.h	41
Timer.cpp	42
Timer.h	42

Глава 3

Алфавитный указатель пространств имен

3.1 Пространства имен

Полный список пространств имен.

<code>plot_distribution</code>	13
<code>plot_timing</code>	14

Глава 4

Алфавитный указатель классов

4.1 Классы

Классы с их кратким описанием.

ModifiedLCG	Генератор псевдослучайных чисел на основе LCG	15
ModifiedMiddleSquare	Генератор, улучшающий классический метод середины квадрата	16
ModifiedXorShift	Генератор на основе алгоритма XorShift128	17
SampleStats	Набор описательных статистик	18
TestResult	Результат выполнения одного теста	20
Timer	Класс для измерения времени выполнения участка кода	20

Глава 5

Список файлов

5.1 Файлы

Полный список файлов.

build/CMakeFiles/4.3.1/CompilerIdC/ apple-sdk.c	23
build/CMakeFiles/4.3.1/CompilerIdC/ CMakeCCompilerId.c	23
build/CMakeFiles/4.3.1/CompilerIdCXX/ apple-sdk.cpp	26
build/CMakeFiles/4.3.1/CompilerIdCXX/ CMakeCXXCompilerId.cpp	26
scripts/ plot_distribution.py	30
scripts/ plot_timing.py	30
src/ Benchmark.h	
Шаблонная функция для измерения времени генерации n чисел	30
src/ main.cpp	32
src/ ModifiedLCG.cpp	33
src/ ModifiedLCG.h	
Линейный конгруэнтный генератор с модификациями	33
src/ ModifiedMiddleSquare.cpp	34
src/ ModifiedMiddleSquare.h	
Модифицированный метод середины квадрата	34
src/ ModifiedXorShift.cpp	35
src/ ModifiedXorShift.h	
Генератор XorShift128 с добавлением нелинейности	35
src/ RandomTests.cpp	36
src/ RandomTests.h	
Набор статистических тестов для случайных последовательностей	37
src/ Statistics.cpp	40
src/ Statistics.h	
Вычисление основных статистик выборки	41
src/ Timer.cpp	
Реализация класса Timer для измерения времени	42
src/ Timer.h	42

Глава 6

Директории

6.1 Содержание директории build/CMakeFiles/4.3.1

Директории

- директория [CompilerIdC](#)
- директория [CompilerIdCXX](#)

6.2 Содержание директории build

Директории

- директория [CMakeFiles](#)

6.3 Содержание директории build/CMakeFiles

Директории

- директория [4.3.1](#)

6.4 Содержание директории build/CMakeFiles/4.3.1/CompilerIdC

Файлы

- файл [apple-sdk.c](#)
- файл [CMakeCCompilerId.c](#)

6.5 Содержание директории build/CMakeFiles/4.3.1/CompilerIdCXX

Файлы

- файл [apple-sdk.cpp](#)
- файл [CMakeCXXCompilerId.cpp](#)

6.6 Содержание директории scripts

Файлы

- файл [plot_distribution.py](#)
- файл [plot_timing.py](#)

6.7 Содержание директории src

Файлы

- файл [Benchmark.h](#)
Шаблонная функция для измерения времени генерации n чисел.
- файл [main.cpp](#)
- файл [ModifiedLCG.cpp](#)
- файл [ModifiedLCG.h](#)
Линейный конгруэнтный генератор с модификациями.
- файл [ModifiedMiddleSquare.cpp](#)
- файл [ModifiedMiddleSquare.h](#)
Модифицированный метод середины квадрата.
- файл [ModifiedXorShift.cpp](#)
- файл [ModifiedXorShift.h](#)
Генератор XorShift128 с добавлением нелинейности.
- файл [RandomTests.cpp](#)
- файл [RandomTests.h](#)
Набор статистических тестов для случайных последовательностей.
- файл [Statistics.cpp](#)
- файл [Statistics.h](#)
Вычисление основных статистик выборки.
- файл [Timer.cpp](#)
Реализация класса [Timer](#) для измерения времени.
- файл [Timer.h](#)

Глава 7

Пространства имен

7.1 Пространство имен plot_distribution

Переменные

- int `n` = 100000
- list `data`
- `figsize`
- `bins`
- `dpi`

7.1.1 Переменные

7.1.1.1 `bins`

`plot_distribution.bins`

7.1.1.2 `data`

list `plot_distribution.data`

Инициализатор

```
00001 = [  
00002     random.random()  
00003     for _ in range(n)  
00004 ]
```

7.1.1.3 `dpi`

`plot_distribution.dpi`

7.1.1.4 `figsize`

`plot_distribution.figsize`

7.1.1.5 n

```
int plot_distribution.n = 100000
```

7.2 Пространство имен plot_timing

Переменные

- `df` = `pd.read_csv("../timing_results.csv")`
- `figsize`
- `subset` = `df[df["generator"] == gen]`
- `marker`
- `label`
- `dpi`

7.2.1 Переменные

7.2.1.1 df

```
plot_timing.df = pd.read_csv("../timing_results.csv")
```

7.2.1.2 dpi

```
plot_timing.dpi
```

7.2.1.3 figsize

```
plot_timing.figsize
```

7.2.1.4 label

```
plot_timing.label
```

7.2.1.5 marker

```
plot_timing.marker
```

7.2.1.6 subset

```
plot_timing.subset = df[df["generator"] == gen]
```

Глава 8

Классы

8.1 Класс ModifiedLCG

Генератор псевдослучайных чисел на основе LCG.

```
#include <ModifiedLCG.h>
```

Открытые члены

- `ModifiedLCG` (`uint64_t` seed)
Конструктор.
- `uint32_t next` ()
Генерирует 32-битное псевдослучайное число.
- `double nextDouble` ()
Генерирует число с плавающей точкой в $[0, 1)$.

8.1.1 Подробное описание

Генератор псевдослучайных чисел на основе LCG.

Использует 64-битное состояние, линейное преобразование и нелинейное перемешивание старших 32 бит.

8.1.2 Конструктор(ы)

8.1.2.1 ModifiedLCG()

```
ModifiedLCG::ModifiedLCG (  
    uint64_t seed) [explicit]
```

Конструктор.

Аргументы

seed	Начальное значение состояния.
------	-------------------------------

8.1.3 Методы

8.1.3.1 next()

`uint32_t ModifiedLCG::next ()`

Генерирует 32-битное псевдослучайное число.

Возвращает

`uint32_t` Случайное число в диапазоне $[0, 2^{32}-1]$.

8.1.3.2 nextDouble()

`double ModifiedLCG::nextDouble ()`

Генерирует число с плавающей точкой в $[0, 1)$.

Возвращает

`double` Случайное число.

Объявления и описания членов классов находятся в файлах:

- [src/ModifiedLCG.h](#)
- [src/ModifiedLCG.cpp](#)

8.2 Класс ModifiedMiddleSquare

Генератор, улучшающий классический метод середины квадрата.

```
#include <ModifiedMiddleSquare.h>
```

Открытые члены

- [ModifiedMiddleSquare](#) (`uint32_t` seed)
Конструктор.
- `uint32_t` [next](#) ()
Генерирует 32-битное число.
- `double` [nextDouble](#) ()
Генерирует `double` в $[0, 1)$.

8.2.1 Подробное описание

Генератор, улучшающий классический метод середины квадрата.

На каждом шаге: $state = state^2$, берутся средние 32 бита, циклический сдвиг, XOR с предыдущим состоянием. Предотвращается вырождение в ноль.

8.2.2 Конструктор(ы)

8.2.2.1 ModifiedMiddleSquare()

```
ModifiedMiddleSquare::ModifiedMiddleSquare (
    uint32_t seed) [explicit]
```

Конструктор.

Аргументы

seed	Начальное состояние (если 0, заменяется на константу).
------	--

8.2.3 Методы

8.2.3.1 next()

```
uint32_t ModifiedMiddleSquare::next ()
```

Генерирует 32-битное число.

8.2.3.2 nextDouble()

```
double ModifiedMiddleSquare::nextDouble ()
```

Генерирует double в [0, 1).

Объявления и описания членов классов находятся в файлах:

- [src/ModifiedMiddleSquare.h](#)
- [src/ModifiedMiddleSquare.cpp](#)

8.3 Класс ModifiedXorShift

Генератор на основе алгоритма XorShift128.

```
#include <ModifiedXorShift.h>
```

Открытые члены

- [ModifiedXorShift](#) (uint32_t seed)
Конструктор.
- uint32_t [next](#) ()
Генерирует 32-битное число.
- double [nextDouble](#) ()
Генерирует число double в [0, 1).

8.3.1 Подробное описание

Генератор на основе алгоритма XorShift128.

Состояние из четырёх 32-битных слов. В конце каждого цикла к выходному значению прибавляется предыдущее состояние `u` для усиления нелинейности.

8.3.2 Конструктор(ы)

8.3.2.1 ModifiedXorShift()

```
ModifiedXorShift::ModifiedXorShift (
    uint32_t seed) [explicit]
```

Конструктор.

Аргументы

seed	Начальное зерно (используется для инициализации всех слов).
------	---

8.3.3 Методы

8.3.3.1 next()

```
uint32_t ModifiedXorShift::next ()
```

Генерирует 32-битное число.

8.3.3.2 nextDouble()

```
double ModifiedXorShift::nextDouble ()
```

Генерирует число `double` в $[0, 1)$.

Объявления и описания членов классов находятся в файлах:

- [src/ModifiedXorShift.h](#)
- [src/ModifiedXorShift.cpp](#)

8.4 Структура SampleStats

Набор описательных статистик.

```
#include <Statistics.h>
```

Открытые атрибуты

- double [mean](#)
Среднее арифметическое
- double [variance](#)
Несмещённая дисперсия (с делением на n-1).
- double [stddev](#)
Среднеквадратическое отклонение
- double [cv](#)
Коэффициент вариации (stddev/mean).

8.4.1 Подробное описание

Набор описательных статистик.

8.4.2 Данные класса

8.4.2.1 cv

double SampleStats::cv

Коэффициент вариации (stddev/mean).

8.4.2.2 mean

double SampleStats::mean

Среднее арифметическое

8.4.2.3 stddev

double SampleStats::stddev

Среднеквадратическое отклонение

8.4.2.4 variance

double SampleStats::variance

Несмещённая дисперсия (с делением на n-1).

Объявления и описания членов структуры находятся в файле:

- [src/Statistics.h](#)

8.5 Структура `TestResult`

Результат выполнения одного теста.

```
#include <RandomTests.h>
```

Открытые атрибуты

- `double` `statistic`
Значение статистики теста (Z , χ^2 и т.д.).
- `double` `pvalue`
р-значение (не вычисляется в текущей версии)
- `bool` `passed`
Прошёл ли тест (true/false).

8.5.1 Подробное описание

Результат выполнения одного теста.

8.5.2 Данные класса

8.5.2.1 `passed`

```
bool TestResult::passed
```

Прошёл ли тест (true/false).

8.5.2.2 `pvalue`

```
double TestResult::pvalue
```

р-значение (не вычисляется в текущей версии)

8.5.2.3 `statistic`

```
double TestResult::statistic
```

Значение статистики теста (Z , χ^2 и т.д.).

Объявления и описания членов структуры находятся в файле:

- `src/RandomTests.h`

8.6 Класс `Timer`

Класс для измерения времени выполнения участка кода.

```
#include <Timer.h>
```

Открытые члены

- [Timer](#) ()
Конструктор – запускает таймер.
- [~Timer](#) ()=default
Деструктор по умолчанию (не делает ничего, так как ресурсов нет).
- double [elapsed](#) () const
Возвращает время, прошедшее с момента создания таймера.

8.6.1 Подробное описание

Класс для измерения времени выполнения участка кода.

Использует `std::chrono::high_resolution_clock` для высокоточных замеров. Время измеряется в миллисекундах (double).

8.6.2 Конструктор(ы)

8.6.2.1 Timer()

`Timer::Timer ()`

Конструктор – запускает таймер.

Фиксирует текущее время в момент создания объекта.

Фиксирует текущее время с помощью `high_resolution_clock`.

8.6.2.2 ~Timer()

`Timer::~~Timer () [default]`

Деструктор по умолчанию (не делает ничего, так как ресурсов нет).

8.6.3 Методы

8.6.3.1 elapsed()

`double Timer::elapsed () const`

Возвращает время, прошедшее с момента создания таймера.

Возвращает

Прошедшее время в миллисекундах (тип double).

Время в миллисекундах (double).

Объявления и описания членов классов находятся в файлах:

- `src/`[Timer.h](#)
- `src/`[Timer.cpp](#)

Глава 9

Файлы

9.1 Файл build/CMakeFiles/4.3.1/CompilerIdC/apple-sdk.c

```
#include <AvailabilityMacros.h>
```

9.2 Файл build/CMakeFiles/4.3.1/CompilerIdC/CMakeCCompilerId.c

Макросы

- `#define __has_include(x)`
- `#define COMPILER_ID ""`
- `#define STRINGIFY_HELPER(X)`
- `#define STRINGIFY(X)`
- `#define PLATFORM_ID`
- `#define ARCHITECTURE_ID`
- `#define DEC(n)`
- `#define HEX(n)`
- `#define C_STD_99 199901L`
- `#define C_STD_11 201112L`
- `#define C_STD_17 201710L`
- `#define C_STD_23 202311L`
- `#define C_VERSION`

Функции

- `int main (int argc, char *argv[])`

Переменные

- `char const * info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"`
- `char const * info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"`
- `char const * info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"`
- `const char * info_language_standard_default`
- `const char * info_language_extensions_default`

9.2.1 Макросы

9.2.1.1 __has_include

```
#define __has_include(  
    x)
```

Макроопределение:

```
0
```

9.2.1.2 ARCHITECTURE_ID

```
#define ARCHITECTURE_ID
```

9.2.1.3 C_STD_11

```
#define C_STD_11 201112L
```

9.2.1.4 C_STD_17

```
#define C_STD_17 201710L
```

9.2.1.5 C_STD_23

```
#define C_STD_23 202311L
```

9.2.1.6 C_STD_99

```
#define C_STD_99 199901L
```

9.2.1.7 C_VERSION

```
#define C_VERSION
```

9.2.1.8 COMPILER_ID

```
#define COMPILER_ID ""
```

9.2.1.9 DEC

```
#define DEC(  
    n)
```

Макроопределение:

```
('0' + (((n) / 10000000)%10)), \
('0' + (((n) / 1000000)%10)), \
('0' + (((n) / 100000)%10)), \
('0' + (((n) / 10000)%10)), \
('0' + (((n) / 1000)%10)), \
('0' + (((n) / 100)%10)), \
('0' + (((n) / 10)%10)), \
('0' + ((n) % 10))
```

9.2.1.10 HEX

```
#define HEX(  
    n)
```

Макроопределение:

```
('0' + ((n)»28 & 0xF)), \  
( '0' + ((n)»24 & 0xF)), \  
( '0' + ((n)»20 & 0xF)), \  
( '0' + ((n)»16 & 0xF)), \  
( '0' + ((n)»12 & 0xF)), \  
( '0' + ((n)»8  & 0xF)), \  
( '0' + ((n)»4  & 0xF)), \  
( '0' + ((n)   & 0xF))
```

9.2.1.11 PLATFORM_ID

```
#define PLATFORM_ID
```

9.2.1.12 STRINGIFY

```
#define STRINGIFY(  
    X)
```

Макроопределение:

```
STRINGIFY_HELPER(X)
```

9.2.1.13 STRINGIFY_HELPER

```
#define STRINGIFY_HELPER(  
    X)
```

Макроопределение:

```
#X
```

9.2.2 Функции

9.2.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

9.2.3 Переменные

9.2.3.1 info_arch

```
char const* info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
```

9.2.3.2 info_compiler

```
char const* info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
```

9.2.3.3 info_language_extensions_default

```
const char* info_language_extensions_default
```

Инициализатор

```
= "INFO" ":" "extensions_default["
```

```
"OFF"
```

```
"]"
```

9.2.3.4 info_language_standard_default

```
const char* info_language_standard_default
```

Инициализатор

```
=  
"INFO" ":" "standard_default[" C_VERSION "]"
```

9.2.3.5 info_platform

```
char const* info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
```

9.3 Файл build/CMakeFiles/4.3.1/CompilerIdCXX/apple-sdk.cpp

```
#include <AvailabilityMacros.h>
```

9.4 Файл

build/CMakeFiles/4.3.1/CompilerIdCXX/CMakeCXXCompilerId.cpp

Макросы

- #define __has_include(x)
- #define COMPILER_ID ""
- #define STRINGIFY_HELPER(X)
- #define STRINGIFY(X)
- #define PLATFORM_ID
- #define ARCHITECTURE_ID
- #define DEC(n)
- #define HEX(n)
- #define CXX_STD_98 199711L
- #define CXX_STD_11 201103L
- #define CXX_STD_14 201402L
- #define CXX_STD_17 201703L
- #define CXX_STD_20 202002L
- #define CXX_STD_23 202302L
- #define CXX_STD __cplusplus

Функции

- int `main` (int argc, char *argv[])

Переменные

- char const * `info_compiler` = "INFO" ":" "compiler[" COMPILER_ID "]"
- char const * `info_platform` = "INFO" ":" "platform[" PLATFORM_ID "]"
- char const * `info_arch` = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
- const char * `info_language_standard_default`
- const char * `info_language_extensions_default`

9.4.1 Макросы

9.4.1.1 `__has_include`

```
#define __has_include(  
    x)
```

Макроопределение:

```
0
```

9.4.1.2 `ARCHITECTURE_ID`

```
#define ARCHITECTURE_ID
```

9.4.1.3 `COMPILER_ID`

```
#define COMPILER_ID ""
```

9.4.1.4 `CXX_STD`

```
#define CXX_STD __cplusplus
```

9.4.1.5 `CXX_STD_11`

```
#define CXX_STD_11 201103L
```

9.4.1.6 `CXX_STD_14`

```
#define CXX_STD_14 201402L
```

9.4.1.7 `CXX_STD_17`

```
#define CXX_STD_17 201703L
```

9.4.1.8 CXX_STD_20

```
#define CXX_STD_20 202002L
```

9.4.1.9 CXX_STD_23

```
#define CXX_STD_23 202302L
```

9.4.1.10 CXX_STD_98

```
#define CXX_STD_98 199711L
```

9.4.1.11 DEC

```
#define DEC(  
    n)
```

Макроопределение:

```
('0' + (((n) / 10000000)%10)), \
('0' + (((n) / 1000000)%10)), \
('0' + (((n) / 100000)%10)), \
('0' + (((n) / 10000)%10)), \
('0' + (((n) / 1000)%10)), \
('0' + (((n) / 100)%10)), \
('0' + (((n) / 10)%10)), \
('0' + ((n) % 10))
```

9.4.1.12 HEX

```
#define HEX(  
    n)
```

Макроопределение:

```
('0' + ((n)>>28 & 0xF)), \
('0' + ((n)>>24 & 0xF)), \
('0' + ((n)>>20 & 0xF)), \
('0' + ((n)>>16 & 0xF)), \
('0' + ((n)>>12 & 0xF)), \
('0' + ((n)>>8 & 0xF)), \
('0' + ((n)>>4 & 0xF)), \
('0' + ((n) & 0xF))
```

9.4.1.13 PLATFORM_ID

```
#define PLATFORM_ID
```

9.4.1.14 STRINGIFY

```
#define STRINGIFY(  
    X)
```

Макроопределение:

```
STRINGIFY_HELPER(X)
```

9.4.1.15 STRINGIFY_HELPER

```
#define STRINGIFY_HELPER(  
    X)
```

Макроопределение:

```
#X
```

9.4.2 Функции

9.4.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

9.4.3 Переменные

9.4.3.1 info_arch

```
char const* info_arch = "INFO" ":" "arch[" ARCHITECTURE_ID "]"
```

9.4.3.2 info_compiler

```
char const* info_compiler = "INFO" ":" "compiler[" COMPILER_ID "]"
```

9.4.3.3 info_language_extensions_default

```
const char* info_language_extensions_default
```

Инициализатор

```
= "INFO" ":" "extensions_default["
```

```
"OFF"
```

```
"]"
```

9.4.3.4 info_language_standard_default

```
const char* info_language_standard_default
```

Инициализатор

```
= "INFO" ":" "standard_default["
```

```
"98"
```

```
"]"
```

9.4.3.5 info_platform

```
char const* info_platform = "INFO" ":" "platform[" PLATFORM_ID "]"
```

9.5 Файл scripts/plot_distribution.py

Пространства имен

- namespace `plot_distribution`

Переменные

- int `plot_distribution.n` = 100000
- list `plot_distribution.data`
- `plot_distribution.figsize`
- `plot_distribution.bins`
- `plot_distribution.dpi`

9.6 Файл scripts/plot_timing.py

Пространства имен

- namespace `plot_timing`

Переменные

- `plot_timing.df` = `pd.read_csv("../timing_results.csv")`
- `plot_timing.figsize`
- `plot_timing.subset` = `df[df["generator"] == gen]`
- `plot_timing.marker`
- `plot_timing.label`
- `plot_timing.dpi`

9.7 Файл src/Benchmark.h

Шаблонная функция для измерения времени генерации `n` чисел.

```
#include "Timer.h"
```

Функции

- `template<typename RNG>`
`double benchmark (RNG &rng, size_t n)`
 Замеряет время, затраченное на генерацию `n` чисел.

9.7.1 Подробное описание

Шаблонная функция для измерения времени генерации n чисел.

9.7.2 Функции

9.7.2.1 benchmark()

```
template<typename RNG>
double benchmark (
    RNG & rng,
    size_t n)
```

Замеряет время, затраченное на генерацию n чисел.

Параметры шаблона

RNG	Тип генератора (должен иметь метод next()).
-----	---

Аргументы

rng	Ссылка на генератор.
n	Количество генерируемых чисел.

Возвращает

double Время в миллисекундах.

9.8 Benchmark.h

[См. документацию.](#)

```
00001
00002
00003 #ifndef BENCHMARK_H
00004 #define BENCHMARK_H
00005
00006 #include "Timer.h"
00007
00008 template<typename RNG>
00009 double benchmark(RNG& rng, size_t n) {
00010     Timer timer;
00011     for (size_t i = 0; i < n; ++i) {
00012         volatile uint32_t x = rng.next();
00013         (void)x;
00014     }
00015     return timer.elapsed();
00016 }
00017
00018 #endif
```

9.9 Файл src/main.cpp

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <vector>
#include <random>
#include "ModifiedLCG.h"
#include "ModifiedXorShift.h"
#include "ModifiedMiddleSquare.h"
#include "Statistics.h"
#include "RandomTests.h"
#include "Benchmark.h"
```

Макросы

- `#define _USE_MATH_DEFINES`

Функции

- `template<typename RNG>`
`std::vector< double > generateSample (RNG &rng, size_t n)`
- `void printHeader ()`
- `template<typename RNG>`
`void testGenerator (const std::string &name, RNG &rng, std::ofstream &timingCsv)`
- `double benchmarkMt19937 (size_t n)`
- `int main ()`

9.9.1 Макросы

9.9.1.1 _USE_MATH_DEFINES

```
#define _USE_MATH_DEFINES
```

9.9.2 Функции

9.9.2.1 benchmarkMt19937()

```
double benchmarkMt19937 (
    size_t n)
```

9.9.2.2 generateSample()

```
template<typename RNG>
std::vector< double > generateSample (
    RNG & rng,
    size_t n)
```

9.9.2.3 main()

```
int main ()
```

9.9.2.4 printHeader()

```
void printHeader ()
```

9.9.2.5 testGenerator()

```
template<typename RNG>
void testGenerator (
    const std::string & name,
    RNG & rng,
    std::ofstream & timingCsv)
```

9.10 Файл src/ModifiedLCG.cpp

```
#include "ModifiedLCG.h"
```

9.11 Файл src/ModifiedLCG.h

Линейный конгруэнтный генератор с модификациями.

```
#include <cstdint>
```

Классы

- class [ModifiedLCG](#)
Генератор псевдослучайных чисел на основе LCG.

9.11.1 Подробное описание

Линейный конгруэнтный генератор с модификациями.

9.12 ModifiedLCG.h

[См. документацию.](#)

```

00001
00005
00006 #ifndef MODIFIED_LCG_H
00007 #define MODIFIED_LCG_H
00008
00009 #include <stdint>
00010
00018 class ModifiedLCG {
00019 private:
00020     uint64_t state;
00021     static constexpr uint64_t a = 6364136223846793005ull;
00022     static constexpr uint64_t c = 1442695040888963407ull;
00023
00024 public:
00029     explicit ModifiedLCG(uint64_t seed);
00030
00035     uint32_t next();
00036
00041     double nextDouble();
00042 };
00043
00044 #endif // MODIFIED_LCG_H

```

9.13 Файл src/ModifiedMiddleSquare.cpp

```
#include "ModifiedMiddleSquare.h"
```

9.14 Файл src/ModifiedMiddleSquare.h

Модифицированный метод середины квадрата.

```
#include <stdint>
```

Классы

- class [ModifiedMiddleSquare](#)

Генератор, улучшающий классический метод середины квадрата.

9.14.1 Подробное описание

Модифицированный метод середины квадрата.

9.15 ModifiedMiddleSquare.h

[См. документацию.](#)

```

00001
00005
00006 #ifndef MODIFIED_MIDDLE_SQUARE_H
00007 #define MODIFIED_MIDDLE_SQUARE_H
00008
00009 #include <stdint>
00010
00018 class ModifiedMiddleSquare {
00019 private:
00020     uint32_t state;
00021
00022 public:
00027     explicit ModifiedMiddleSquare(uint32_t seed);
00028
00030     uint32_t next();
00031
00033     double nextDouble();
00034 };
00035
00036 #endif // MODIFIED_MIDDLE_SQUARE_H

```

9.16 Файл src/ModifiedXorShift.cpp

```
#include "ModifiedXorShift.h"
```

9.17 Файл src/ModifiedXorShift.h

Генератор XorShift128 с добавлением нелинейности.

```
#include <stdint>
```

Классы

- class [ModifiedXorShift](#)
Генератор на основе алгоритма XorShift128.

9.17.1 Подробное описание

Генератор XorShift128 с добавлением нелинейности.

9.18 ModifiedXorShift.h

[См. документацию.](#)

```

00001
00005 #ifndef MODIFIED_XORSHIFT_H
00006 #define MODIFIED_XORSHIFT_H
00007
00008 #include <stdint>
00009
00017 class ModifiedXorShift {
00018 private:
00019     uint32_t x, y, z, w;
00020
00021 public:
00026     explicit ModifiedXorShift(uint32_t seed);
00027
00029     uint32_t next();
00030
00032     double nextDouble();
00033 };
00034
00035 #endif // MODIFIED_XORSHIFT_H

```

9.19 Файл src/RandomTests.cpp

```
#include "RandomTests.h"
#include <cmath>
#include <map>
#include <bitset>
```

Функции

- **TestResult chiSquareTest** (const std::vector< double > &data)
Тест хи-квадрат на равномерность распределения.
- **TestResult runsTest** (const std::vector< double > &data)
Тест серий (runs test) для проверки независимости битов.
- **TestResult overlappingSumsTest** (const std::vector< double > &data)
Тест перекрывающихся сумм (overlapping sums).
- **TestResult minimumDistanceTest** (const std::vector< double > &data)
Тест минимального расстояния между точками на единичном квадрате.
- **TestResult monkeyTest** (const std::vector< double > &data)
Тест "обезьяны" – проверка покрытия 5-битовых слов.

9.19.1 Функции

9.19.1.1 chiSquareTest()

TestResult chiSquareTest (
const std::vector< double > & data)

Тест хи-квадрат на равномерность распределения.

9.19.1.2 minimumDistanceTest()

TestResult minimumDistanceTest (
const std::vector< double > & data)

Тест минимального расстояния между точками на единичном квадрате.

9.19.1.3 monkeyTest()

TestResult monkeyTest (
const std::vector< double > & data)

Тест "обезьяны" – проверка покрытия 5-битовых слов.

Аргументы

data	Входная последовательность чисел double.
------	--

Возвращает

Результат: passed, если пропущено менее 5 слов из 32.

9.19.1.4 overlappingSumsTest()

```
TestResult overlappingSumsTest (
    const std::vector< double > & data)
```

Тест перекрывающихся сумм (overlapping sums).

9.19.1.5 runsTest()

```
TestResult runsTest (
    const std::vector< double > & data)
```

Тест серий (runs test) для проверки независимости битов.

9.20 Файл src/RandomTests.h

Набор статистических тестов для случайных последовательностей.

```
#include <vector>
#include "RandomTests.hpp"
```

Классы

- struct `TestResult`
Результат выполнения одного теста.

Функции

- `TestResult chiSquareTest` (const std::vector< double > &data)
Тест хи-квадрат на равномерность распределения.
- `TestResult runsTest` (const std::vector< double > &data)
Тест серий (runs test) для проверки независимости битов.
- `TestResult overlappingSumsTest` (const std::vector< double > &data)
Тест перекрывающихся сумм (overlapping sums).
- `TestResult minimumDistanceTest` (const std::vector< double > &data)
Тест минимального расстояния между точками на единичном квадрате.
- `TestResult monkeyTest` (const std::vector< double > &data)
Тест "обезьяны" – проверка покрытия 5-битовых слов.
- template<typename RNG>
`TestResult squeezeTest` (RNG &rng, int repetitions)
Тест "сжатия" (squeeze) – моделирует многократное умножение.

9.20.1 Подробное описание

Набор статистических тестов для случайных последовательностей.

9.20.2 Функции

9.20.2.1 chiSquareTest()

```
TestResult chiSquareTest (  
    const std::vector< double > & data)
```

Тест хи-квадрат на равномерность распределения.

9.20.2.2 minimumDistanceTest()

```
TestResult minimumDistanceTest (  
    const std::vector< double > & data)
```

Тест минимального расстояния между точками на единичном квадрате.

9.20.2.3 monkeyTest()

```
TestResult monkeyTest (  
    const std::vector< double > & data)
```

Тест "обезьяны" – проверка покрытия 5-битовых слов.

Аргументы

data	Входная последовательность чисел double.
------	--

Возвращает

Результат: passed, если пропущено менее 5 слов из 32.

9.20.2.4 overlappingSumsTest()

```
TestResult overlappingSumsTest (  
    const std::vector< double > & data)
```

Тест перекрывающихся сумм (overlapping sums).

9.20.2.5 runsTest()

```
TestResult runsTest (  
    const std::vector< double > & data)
```

Тест серий (runs test) для проверки независимости битов.

9.20.2.6 squeezeTest()

```
template<typename RNG>
TestResult squeezeTest (
    RNG & rng,
    int repetitions)
```

Тест "сжатия" (squeeze) – моделирует многократное умножение.

Параметры шаблона

RNG	Тип генератора.
-----	-----------------

Аргументы

rng	Генератор случайных чисел.
-----	----------------------------

repetitions	Количество повторений эксперимента.
-------------	-------------------------------------

Возвращает

Результат: passed, если среднее число шагов близко к 21.

9.21 RandomTests.h

[См. документацию.](#)

```

00001
00005
00006 #ifndef RANDOM_TESTS_H
00007 #define RANDOM_TESTS_H
00008
00009 #include <vector>
00010
00015 struct TestResult {
00016     double statistic;
00017     double pvalue;
00018     bool passed;
00019 };
00020
00022 TestResult chiSquareTest(const std::vector<double>& data);
00023
00025 TestResult runsTest(const std::vector<double>& data);
00026
00028 TestResult overlappingSumsTest(const std::vector<double>& data);
00029
00031 TestResult minimumDistanceTest(const std::vector<double>& data);
00032
00038 TestResult monkeyTest(const std::vector<double>& data);
00039
00047 template<typename RNG>
00048 TestResult squeezeTest(RNG& rng, int repetitions);
00049
00050 #include "RandomTests.tpp"
00051
00052 #endif // RANDOM_TESTS_H

```

9.22 Файл src/Statistics.cpp

```

#include "Statistics.h"
#include <cmath>

```

Функции

- [SampleStats computeStats](#) (const std::vector< double > &data)
Вычисляет статистики для выборки.

9.22.1 Функции

9.22.1.1 computeStats()

`SampleStats` computeStats (
 const std::vector< double > & data)

Вычисляет статистики для выборки.

Аргументы

data	Входной вектор вещественных чисел.
------	------------------------------------

Возвращает

`SampleStats` Структура с рассчитанными значениями.

9.23 Файл src/Statistics.h

Вычисление основных статистик выборки.

```
#include <vector>
```

Классы

- struct `SampleStats`
 Набор описательных статистик.

Функции

- `SampleStats` computeStats (const std::vector< double > &data)
 Вычисляет статистики для выборки.

9.23.1 Подробное описание

Вычисление основных статистик выборки.

9.23.2 Функции

9.23.2.1 computeStats()

`SampleStats` computeStats (
 const std::vector< double > & data)

Вычисляет статистики для выборки.

Аргументы

data	Входной вектор вещественных чисел.
------	------------------------------------

Возвращает

`SampleStats` Структура с рассчитанными значениями.

9.24 Statistics.h

[См. документацию.](#)

```
00001
00005
00006 #ifndef STATISTICS_H
00007 #define STATISTICS_H
00008
00009 #include <vector>
00010
00015 struct SampleStats {
00016     double mean;
00017     double variance;
00018     double stddev;
00019     double cv;
00020 };
00021
00027 SampleStats computeStats(const std::vector<double>& data);
00028
00029 #endif
```

9.25 Файл src/Timer.cpp

Реализация класса [Timer](#) для измерения времени.

```
#include "Timer.h"
#include <iostream>
#include <iomanip>
```

9.25.1 Подробное описание

Реализация класса [Timer](#) для измерения времени.

9.26 Файл src/Timer.h

```
#include <chrono>
```

Классы

- class [Timer](#)

Класс для измерения времени выполнения участка кода.

9.27 Timer.h

[См. документацию.](#)

```
00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00012 class Timer {
00013
00014 private:
00015     std::chrono::time_point<std::chrono::steady_clock> start;
00016
00017 public:
00023     Timer();
00027     ~Timer() = default;
00032     double elapsed() const;
00033 };
00034
00035 #endif
```

Предметный указатель

- `_USE_MATH_DEFINES`
 - `main.cpp`, [32](#)
 - `__has_include`
 - `CMakeCCompilerId.c`, [24](#)
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `~Timer`
 - `Timer`, [21](#)
- `ARCHITECTURE_ID`
 - `CMakeCCompilerId.c`, [24](#)
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `benchmark`
 - `Benchmark.h`, [31](#)
- `Benchmark.h`
 - `benchmark`, [31](#)
- `benchmarkMt19937`
 - `main.cpp`, [32](#)
- `bins`
 - `plot_distribution`, [13](#)
- `build/CMakeFiles/4.3.1/CompilerIdC/apple-sdk.c`,
[23](#)
- `build/CMakeFiles/4.3.1/CompilerIdC/CMakeCCompilerId.c`,
[23](#)
- `build/CMakeFiles/4.3.1/CompilerIdCXX/apple-sdk.cpp`, [26](#)
- `build/CMakeFiles/4.3.1/CompilerIdCXX/CMakeCXXCompilerId.cpp`,
[26](#)
- `C_STD_11`
 - `CMakeCCompilerId.c`, [24](#)
- `C_STD_17`
 - `CMakeCCompilerId.c`, [24](#)
- `C_STD_23`
 - `CMakeCCompilerId.c`, [24](#)
- `C_STD_99`
 - `CMakeCCompilerId.c`, [24](#)
- `C_VERSION`
 - `CMakeCCompilerId.c`, [24](#)
- `chiSquareTest`
 - `RandomTests.cpp`, [36](#)
 - `RandomTests.h`, [38](#)
- `CMakeCCompilerId.c`
 - `__has_include`, [24](#)
 - `ARCHITECTURE_ID`, [24](#)
 - `C_STD_11`, [24](#)
 - `C_STD_17`, [24](#)
 - `C_STD_23`, [24](#)
 - `C_STD_99`, [24](#)
 - `C_VERSION`, [24](#)
 - `COMPILER_ID`, [24](#)
 - `DEC`, [24](#)
 - `HEX`, [24](#)
 - `info_arch`, [25](#)
 - `info_compiler`, [25](#)
 - `info_language_extensions_default`, [26](#)
 - `info_language_standard_default`, [26](#)
 - `info_platform`, [26](#)
 - `main`, [25](#)
 - `PLATFORM_ID`, [25](#)
 - `STRINGIFY`, [25](#)
 - `STRINGIFY_HELPER`, [25](#)
- `CMakeCXXCompilerId.cpp`
 - `__has_include`, [27](#)
 - `ARCHITECTURE_ID`, [27](#)
 - `COMPILER_ID`, [27](#)
 - `CXX_STD`, [27](#)
 - `CXX_STD_11`, [27](#)
 - `CXX_STD_14`, [27](#)
 - `CXX_STD_17`, [27](#)
 - `CXX_STD_20`, [27](#)
 - `CXX_STD_23`, [28](#)
 - `CXX_STD_98`, [28](#)
 - `DEC`, [28](#)
 - `HEX`, [28](#)
 - `info_arch`, [29](#)
 - `info_compiler`, [29](#)
 - `info_language_extensions_default`, [29](#)
 - `info_language_standard_default`, [29](#)
 - `info_platform`, [29](#)
 - `main`, [29](#)
 - `PLATFORM_ID`, [28](#)
 - `STRINGIFY`, [28](#)
 - `STRINGIFY_HELPER`, [28](#)
- `COMPILER_ID`
 - `CMakeCCompilerId.c`, [24](#)
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `computeStats`
 - `Statistics.cpp`, [41](#)
 - `Statistics.h`, [41](#)
- `cv`
 - `SampleStats`, [19](#)
- `CXX_STD`
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `CXX_STD_11`
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `CXX_STD_14`
 - `CMakeCXXCompilerId.cpp`, [27](#)
- `CXX_STD_17`

- CMakeCXXCompilerId.cpp, 27
- CXX_STD_20
 - CMakeCXXCompilerId.cpp, 27
- CXX_STD_23
 - CMakeCXXCompilerId.cpp, 28
- CXX_STD_98
 - CMakeCXXCompilerId.cpp, 28
- data
 - plot_distribution, 13
- DEC
 - CMakeCCompilerId.c, 24
 - CMakeCXXCompilerId.cpp, 28
- df
 - plot_timing, 14
- dpi
 - plot_distribution, 13
 - plot_timing, 14
- elapsed
 - Timer, 21
- figsize
 - plot_distribution, 13
 - plot_timing, 14
- generateSample
 - main.cpp, 32
- HEX
 - CMakeCCompilerId.c, 24
 - CMakeCXXCompilerId.cpp, 28
- info_arch
 - CMakeCCompilerId.c, 25
 - CMakeCXXCompilerId.cpp, 29
- info_compiler
 - CMakeCCompilerId.c, 25
 - CMakeCXXCompilerId.cpp, 29
- info_language_extensions_default
 - CMakeCCompilerId.c, 26
 - CMakeCXXCompilerId.cpp, 29
- info_language_standard_default
 - CMakeCCompilerId.c, 26
 - CMakeCXXCompilerId.cpp, 29
- info_platform
 - CMakeCCompilerId.c, 26
 - CMakeCXXCompilerId.cpp, 29
- label
 - plot_timing, 14
- main
 - CMakeCCompilerId.c, 25
 - CMakeCXXCompilerId.cpp, 29
 - main.cpp, 32
- main.cpp
 - _USE_MATH_DEFINES, 32
 - benchmarkMt19937, 32
 - generateSample, 32
 - main, 32
 - printHeader, 33
 - testGenerator, 33
- marker
 - plot_timing, 14
- mean
 - SampleStats, 19
- minimumDistanceTest
 - RandomTests.cpp, 36
 - RandomTests.h, 38
- ModifiedLCG, 15
 - ModifiedLCG, 15
 - next, 16
 - nextDouble, 16
- ModifiedMiddleSquare, 16
 - ModifiedMiddleSquare, 17
 - next, 17
 - nextDouble, 17
- ModifiedXorShift, 17
 - ModifiedXorShift, 18
 - next, 18
 - nextDouble, 18
- monkeyTest
 - RandomTests.cpp, 36
 - RandomTests.h, 38
- n
 - plot_distribution, 13
- next
 - ModifiedLCG, 16
 - ModifiedMiddleSquare, 17
 - ModifiedXorShift, 18
- nextDouble
 - ModifiedLCG, 16
 - ModifiedMiddleSquare, 17
 - ModifiedXorShift, 18
- overlappingSumsTest
 - RandomTests.cpp, 36
 - RandomTests.h, 38
- passed
 - TestResult, 20
- PLATFORM_ID
 - CMakeCCompilerId.c, 25
 - CMakeCXXCompilerId.cpp, 28
- plot_distribution, 13
 - bins, 13
 - data, 13
 - dpi, 13
 - figsize, 13
 - n, 13
- plot_timing, 14
 - df, 14
 - dpi, 14
 - figsize, 14
 - label, 14
 - marker, 14
 - subset, 14

printHeader
 main.cpp, 33
 pvalue
 TestResult, 20
 RandomTests.cpp
 chiSquareTest, 36
 minimumDistanceTest, 36
 monkeyTest, 36
 overlappingSumsTest, 36
 runsTest, 37
 RandomTests.h
 chiSquareTest, 38
 minimumDistanceTest, 38
 monkeyTest, 38
 overlappingSumsTest, 38
 runsTest, 38
 squeezeTest, 38
 runsTest
 RandomTests.cpp, 37
 RandomTests.h, 38
 SampleStats, 18
 cv, 19
 mean, 19
 stddev, 19
 variance, 19
 scripts/plot_distribution.py, 30
 scripts/plot_timing.py, 30
 squeezeTest
 RandomTests.h, 38
 src/Benchmark.h, 30, 31
 src/main.cpp, 32
 src/ModifiedLCG.cpp, 33
 src/ModifiedLCG.h, 33, 34
 src/ModifiedMiddleSquare.cpp, 34
 src/ModifiedMiddleSquare.h, 34, 35
 src/ModifiedXorShift.cpp, 35
 src/ModifiedXorShift.h, 35
 src/RandomTests.cpp, 36
 src/RandomTests.h, 37, 40
 src/Statistics.cpp, 40
 src/Statistics.h, 41, 42
 src/Timer.cpp, 42
 src/Timer.h, 42
 statistic
 TestResult, 20
 Statistics.cpp
 computeStats, 41
 Statistics.h
 computeStats, 41
 stddev
 SampleStats, 19
 STRINGIFY
 CMakeCCompilerId.c, 25
 CMakeCXXCompilerId.cpp, 28
 STRINGIFY_HELPER
 CMakeCCompilerId.c, 25
 CMakeCXXCompilerId.cpp, 28

subset
 plot_timing, 14
 testGenerator
 main.cpp, 33
 TestResult, 20
 passed, 20
 pvalue, 20
 statistic, 20
 Timer, 20
 ~Timer, 21
 elapsed, 21
 Timer, 21
 variance
 SampleStats, 19
 Содержание директории build, 11
 Содержание директории build/CMakeFiles, 11
 Содержание директории build/CMakeFiles/4.3.1,
 11
 Содержание директории build/CMakeFiles/4.3.1/CompilerIdC,
 11
 Содержание директории build/CMakeFiles/4.3.1/CompilerIdCXX,
 11
 Содержание директории scripts, 12
 Содержание директории src, 12
 Сравнение модифицированных генераторов слу-
 чайных чисел, 1